Mobile Platforms		3 + (2 DM) + (1 GIT) + (1 HCI) + (1 NC)
Robot Platforms		4 + (3 GIT) + (3 AI)
Embedded Platforms		4 + (4 AR) + (1 FPL) + (1 GIT) + (3 OS) + (1 SF)
Game Platforms		4 + (1 AL) + (1 AI) + (4 AR) + (5 GIT) + (2 HCI) + (1 SDF) + (1 SE) + (1 MSF)
Interactive Computing Platforms		3 + (1 DM) + (2 GIT) + (1 AR) + (1 FPL)
SPD-SEP	Included in SEP hours	
Total	**4**	

Note: The CS Core hours total includes 1 hour shared with SE.

Knowledge Units

SPD-Common: Common Aspects/Shared Concerns

CS Core:

1. Overview of development platforms (i.e., web, mobile, game, robotics, embedded, and interactive).
    a. Input/sensors/control devices/haptic devices
    b. Resource constraints
        i. Computational
        ii. Data storage
        iii. Memory
        iv. Communication
    c. Requirements – security, uptime availability, fault tolerance (See also: SE-Reliability, SEC-Engineering)
    d. Output/actuators/haptic devices
2. Programming via platform-specific Application Programming Interface (API) vs traditional application construction
3. Overview of platform Languages (e.g., Python, Swift, Lua, Kotlin)
4. Programming under platform constraints and requirements (e.g., available development tools, development, security considerations) (See also: SEC-Foundations)
5. Techniques for learning and mastering a platform-specific programming language

Illustrative Learning Outcomes:

CS Core:

1. List the constraints of mobile programming.
2. List the characteristics of scripting languages.
3. Describe the three-tier model of web programming.

4. Describe how the state is maintained in web programming.

SPD-Web: Web Platforms

KA Core:

1. Web programming languages (e.g., HTML5, JavaScript, PHP, CSS)
2. Web platforms, frameworks, or meta-frameworks
    a. Cloud services
    b. API, Web Components
3. Software as a Service (SaaS)
4. Web standards such as document object model, accessibility (See also: HCI-Accessibility)
5. Security and Privacy Considerations (See also: SEP-Security)

Non-core:

6. Analyzing requirements for web applications
7. Computing services (See also: DM-NoSQL)
    a. Cloud Hosting
    b. Scalability (e.g., Autoscaling, Clusters)
    c. Cost estimation for services
8. Data management (See also: DM-Core)
    a. Data residency: where the data is located and what paths can be taken to access it
    b. Data integrity: guaranteeing data is accessible and that data is deleted when required
9. Architecture
    a. Monoliths vs Microservices
    b. Micro-frontends
    c. Event-Driven vs RESTful architectures: advantages and disadvantages
    d. Serverless, cloud computing on demand
10. Storage solutions (See also: DM-Relational, DM-NoSQL)
    a. Relational Databases
    b. NoSQL databases

Illustrative Learning Outcomes:

KA Core:

1. Design and implement a web-based application using a microservice architecture design.
2. Describe the constraints, such as hosting, services, and scalability, related to web platforms.
3. Compare and contrast web programming with general-purpose programming.
4. Describe the differences between Software-as-a-Service (SaaS) and traditional software products.
5. Discuss how web standards impact software development.
6. Evaluate an existing web application against current web standards.

SPD-Mobile: Mobile Platforms

KA Core:

1. Development with
    a. Mobile programming languages

    b. Mobile programming environments
2. Mobile platform constraints
    a. User interface design (See also: HCI-User)
    b. Security
3. Access
    a. Accessing data through APIs (See also: DM-Querying)
    b. Designing API endpoints for mobile apps: pitfalls and design considerations
    c. Network and web interfaces (See also: NC-Fundamentals, DM-Modeling)

Non-core:

4. Development
    a. Native versus cross-platform development
    b. Software design/architecture patterns for mobile applications (See also: SE-Design)
5. Mobile platform constraints
    a. Responsive user interface design (See also: HCI-Accessibility)
    b. Heterogeneity and mobility of devices
    c. Differences in user experiences (e.g., between mobile and web-based applications)
    d. Power and performance tradeoff
6. Mobile computing affordances
    a. Location-aware applications
    b. Sensor-driven computing (e.g., gyroscope, accelerometer, health data from a watch)
    c. Telephony and instant messaging
    d. Augmented reality (See also: GIT-Immersion)
7. Specification and testing (See also: SDF-Practices, SE-Validation)
8. Asynchronous computing (See also: PDC-Algorithms)
    a. Difference from traditional synchronous programming
    b. Handling success via callbacks
    c. Handling errors asynchronously
    d. Testing asynchronous code and typical problems in testing

Illustrative Learning Outcomes:

KA Core:

1. Compare mobile programming with general-purpose programming.
2. Develop a location-aware mobile application with data API integration.
3. Build a sensor-driven mobile application capable of logging data on a remote server.
4. Create a communication app incorporating telephony and instant messaging.
5. Evaluate the pros and cons of native and cross-platform mobile application development.

SPD-Robot: Robot Platforms

KA Core:

1. Types of robotic platforms and devices (See also: AI-Robotics)
2. Sensors, embedded computation, and effectors (actuators) (See also: GIT-Physical)
3. Robot-specific languages and libraries (See also: AI-Robotics)
4. Robotic software architecture (e.g., using the Robot Operating System (ROS))

5. Robotic platform constraints and design considerations (See also: AI-Robotics)
6. Interconnections with physical or simulated systems (See also: GIT-Physical, GIT-Simulation)
7. Robotic Algorithms (See also: AI-Robotics, GIT-Animation)
    a. Forward kinematics
    b. Inverse kinematics
    c. Dynamics
    d. Navigation and path planning
    e. Grasping and manipulation
8. Safety and interaction considerations (See also: SEP-Professional-Ethics, SEP-Context)

Illustrative Learning Outcomes:

KA Core:

1. Design and implement an application on a given robotic platform.
2. Integrate an Arduino-based robot kit and program it to navigate a maze.
3. Compare robot-specific languages and techniques with those used for general-purpose software development.
4. Explain the rationale behind the design of the robotic platform and its interconnections with physical or simulated systems.
5. Given a high-level application, design a robot software architecture using ROS specifying all components and interconnections (ROS topics) to accomplish that application.
6. Discuss the constraints a given robotic platform imposes on developers.

SPD-Embedded: Embedded Platforms

KA Core:

1. Introduction to the unique characteristics of embedded systems
    a. Real-time vs soft real-time and non-real-time systems
    b. Resource constraints, such as memory profiles and deadlines (See also: AR-Memory)
2. API for custom architectures
    a. GPU technology (See also: AR-Heterogeneity, GIT-Shading)
    b. Field Programmable Gate Arrays (FPGA) (See also: AR-Logic)
    c. Cross-platform systems
3. Embedded Systems
    a. Microcontrollers
    b. Interrupts and feedback
    c. Interrupt handlers in high-level languages (See also: SF-Overview)
    d. Hard and soft interrupts and trap-exits (See also: OS-Principles)
    e. Interacting with hardware, actuators, and sensors
    f. Energy efficiency
    g. Loosely timed coding and synchronization
    h. Software adapters
4. Embedded programming
5. Hard real-time systems vs soft real-time systems (See also: OS-Real-time)
    a. Timeliness
    b. Time synchronization/scheduling

    c. Prioritization
    d. Latency
    e. Compute jitter
6. Real-time resource management
7. Memory management
    a. Mapping programming construct (variable) to a memory location (See also: AR-Memory)
    b. Shared memory (See also: OS-Memory)
    c. Manual memory management.
    d. Garbage collection (See also: FPL-Translation)
8. Safety considerations and safety analysis (See also: SEP-Context, SEP-Professional-Ethics)
9. Sensors and actuators
10. Analysis and verification
11. Application design

Illustrative Learning Outcomes:

KA Core:

1. Design and implement a small embedded system for a given platform (e.g., a smart alarm clock or a drone).
2. Describe the unique characteristics of embedded systems versus other systems.
3. Interface with sensors/actuators.
4. Debug a problem with an existing embedded platform.
5. Identify different types of embedded architectures.
6. Evaluate which architecture is best for a given set of requirements.
7. Design and develop software to interact with and control hardware.
8. Design methods for real-time systems.
9. Evaluate real-time scheduling and schedulability analysis.
10. Evaluate formal specification and verification of timing constraints and properties.

SPD-Game: Game Platforms

KA Core:

1. Historical and contemporary platforms for games (See also: AR-Logic)
    a. Evolution of Game Platforms (e.g., Brown Box to Metaverse and beyond; Improvement in Computing Architectures (CPU and GPU); Platform Convergence and Mobility)
    b. Typical Game Platforms (e.g., Personal Computer; Home Console; Handheld Console; Arcade Machine; Interactive Television; Mobile Phone; Tablet; Integrated Head-Mounted Display; Immersive Installations and Simulators; Internet of Things enabled Devices; CAVE Systems; Web Browsers; Cloud-based Streaming Systems)
    c. Characteristics and Constraints of Different Game Platforms (e.g., Features (local storage, internetworking, peripherals); Run-time performance (GPU/CPU frequency, number of cores); Chipsets (physics processing units, vector co-processors); Expansion Bandwidth (PCIe); Network throughput (Ethernet); Memory types and capacities (DDR/GDDR); Maximum stack depth; Power consumption; Thermal design; Endian)
    d. Typical Sensors, Controllers, and Actuators (e.g., distinctive control system designs – peripherals (mouse, keypad, joystick), game controllers, wearables, interactive surfaces;

electronics and bespoke hardware; computer vision, inside-out tracking, and outside-in tracking; IoT-enabled electronics and I/O (See also: GIT-Interaction)
    e. eSports Ecosystems (e.g., evolution of gameplay across platforms; games and eSports; game events such as LAN/arcade tournaments and international events such as the Olympic eSports Series; streamed media and spectatorship; multimedia technologies and broadcast management; professional play; data and machine learning for coaching and training)
2. Real-time Simulation and Rendering Systems
    a. CPU and GPU architectures (e.g., Flynn's taxonomy; parallelization; instruction sets; standard components – graphics compute array, graphics memory controller, video graphics array basic input/output system; bus interface; power management unit; video processing unit; display interface) (See also: AR-Heterogeneity)
    b. Pipelines for physical simulations and graphical rendering: (e.g., tile-based, immediate-mode). (See also: GIT-Rendering)
    c. Common Contexts for Algorithms, Data Structures, and Mathematical Functions (e.g., game loops; spatial partitioning, viewport culling, and level of detail; collision detection and resolution; physical simulation; behavior for intelligent agents; procedural content generation) (See also: MSF-Discrete, AL-Foundational)
    d. Media representations (e.g., I/O, and computation techniques for virtual worlds: audio; music; sprites; models and textures; text; dialogue; multimedia (e.g., olfaction, tactile) (See also: GIT-Fundamentals)
3. Game Development Tools and Techniques
    a. Programming Languages (e.g., C++; C#; Lua; Python; JavaScript)
    b. Shader Languages (e.g., HLSL, GLSL; Shader Graph)
    c. Graphics Libraries and APIs (e.g., DirectX; SDL; OpenGL; Metal; Vulkan; WebGL). (See also: GIT-Rendering, HCI-Design)
    d. Common Development Tools and Environments (e.g., IDEs; Debuggers; Profilers; Version Control Systems including those handling binary assets; Development Kits and Production/Consumer Kits; Emulators) (See also: SDF-Practices, SE-Tools)
4. Game Engines
    a. Open Game Engines (e.g., Unreal; Unity; Godot; CryEngine; Phyre; Source 2; Pygame and Ren'Py; Phaser; Twine; Spring RTS)
    b. Techniques (e.g., Ideation, Prototyping, Iterative Design and Implementation, Compiling Executable Builds, Development Operations and Quality Assurance – Play Testing and Technical Testing, Profiling; Optimization, Porting; Internationalization and Localization, Networking) (See also: AR-Performance-Energy, SE-Requirements)
5. Game Design
    a. Vocabulary (e.g., game definitions; mechanics-dynamics-aesthetics model; industry terminology; experience design; models of experience and emotion)
    b. Design Thinking and User-Centered Experience Design (e.g., methods of designing games; iteration, incrementing, and the double-diamond; phases of pre- and post-production; quality assurance, including alpha and beta testing; stakeholder and customer involvement; community management) (See also: SE-Design)

c. Genres (e.g., adventure; walking simulator; first-person shooter; real-time strategy; multiplayer online battle arena (MOBA); role-playing game (rpg))
d. Audiences and Player Taxonomies (e.g., people who play games; diversity and broadening participation; pleasures, player types, and preferences; Bartle, yee) (See also: HCI-User)
e. Proliferation of digital game technologies to domains beyond entertainment (e.g., Education and Training; Serious Games; Virtual Production; eSports; Gamification; Immersive Experience Design; Creative Industry Practice; Artistic Practice; Procedural Rhetoric) (See also: AI-SEP)

Illustrative Learning Outcomes:

KA Core:

1. Recall the characteristics of common general-purpose graphics processing architectures.
2. Identify the key stages of the immediate-mode rendering pipeline.
3. Describe the key constraints a specific game platform will likely impose on developers.
4. Explain how eSports are streamed to large audiences over the internet.
5. Translate complex mathematical functions into performant source code.
6. Use an industry-standard graphics API to render a 3D model in a virtual scene.
7. Modify a shader to change a visual effect according to stated requirements.
8. Implement a game for a particular platform according to the specification.
9. Optimize a function for processing collision detection in a simulated environment.
10. Assess a game's run-time and memory performance using an industry-standard tool and development environment.
11. Compare the interfaces of different game platforms, highlighting their respective implications for human-computer interaction.
12. Recommend an appropriate set of development tools and techniques for implementing a game of a particular genre for a given platform.
13. Discuss the key challenges in making a digital game that is cross-platform compatible.
14. Express how game developers can enhance the accessibility of a game interface.
15. Create novel forms of gameplay using frontier game platforms.

SPD-Interactive: Interactive Computing Platforms

Non-core:

1. Data Analysis Platforms
    a. Jupyter notebooks; Google Colab; R; SPSS; Observable.
    b. Cloud SQL/data analysis platforms (e.g., BigQuery) (See also: DM-Querying)
        i. Apache Spark
        ii. Data Visualizations (See also: GIT-Visualization)
    c. Interactive presentations backed by data
    d. Design tools requiring low-latency feedback loops
        i. Rendering tools
        ii. Graphic design tools
2. Prompt programming
    a. Generative AI (e.g., OpenAI's ChatGPT, OpenAI's Codex, GitHub's Copilot) and LLMs are accessed/interacted

3. Quantum Platforms (See also: AR-Quantum)
    a. Program quantum logic operators in quantum machines.
    b. Use API for available quantum services
    c. Signal analysis/Fourier analysis/Signal processing (for music composition, audio/RF analysis) (See also: GIT-Image)

Illustrative Learning Outcomes:
Non-core:
1. Analyze large datasets interactively.
2. Create a backing track for a musical performance, such as live coding.
3. Create compelling computational notebooks that construct a narrative for a given journalistic goal/story.
4. Implement interactive code that uses a dataset and generates exploratory graphics.
5. Create a program that performs a task using LLM systems.
6. Contrast a program developed by an AI platform and by a human.
7. Implement a system that interacts with a human without using a screen.
8. Contextualize the attributes of different data analysis styles, such as interactive vs engineered pipeline.
9. Write a program using a notebook computing platform (e.g., searching, sorting, or graph manipulation).
10. Demonstrate a quantum gate outcome using a quantum platform.

SPD-SEP/Mobile

Non-core:
1. Privacy and data protection
2. Accessibility in mobile design
3. Security and cybersecurity
4. Social impacts of mobile technology
5. Ethical use of AI and algorithms

Illustrative Learning Outcomes:
Non-core:
1. Understand and uphold ethical responsibilities for safeguarding user privacy and data protection in mobile applications.
2. Design mobile applications with accessibility in mind, ensuring effective use by people with disabilities.
3. Demonstrate proficiency in secure coding practices to mitigate risks associated with various security threats in mobile development.
4. Analyze the broader social impacts of mobile technology, including its influence on communication patterns, relationships, and mental health.
5. Comprehend the ethical considerations of using AI in mobile applications, ensuring unbiased and fair algorithms.

SPD-SEP/Web

Non-core:

1. Privacy concerns with mobile apps
2. Designing for inclusivity and accessibility
3. Ethical use of AI in mobile apps
4. Sustainable app development and server hosting
5. Avoiding spam or intrusive notifications
6. Addressing cyberbullying and harassment
7. Promoting positive online communities
8. Monetization and advertising
9. Ethical use of gamification

Illustrative Learning Outcomes:

Non-core:

1. Understand how mobile computing impacts communications and the flow of information within society.
2. Design mobile apps that have made daily tasks easier/faster.
3. Recognize how the ubiquity of mobile computing has affected work-life balance.
4. Understand how mobile computing impacts health monitoring and healthcare services.
5. Define how mobile apps are used to educate about and help achieve UN sustainability goals.

SPD-SEP/Game

Non-core:

1. Intellectual Property Rights in Creative Industries
    a. Intellectual Property Ownership: copyright, trademark; design right, patent, trade secret, civil versus criminal law; international agreements; procedural content generation and the implications of generative artificial intelligence
    b. Licensing: Usage and fair usage exceptions; open-source license agreements; proprietary and bespoke licensing; enforcement
2. Fair Access to Play
    a. Game Interface Usability: user requirements, affordances, ergonomic design, user research, experience measurement, and heuristic evaluation methods for games
    b. Game Interface Accessibility: forms of impairment and disability; means to facilitate game access; universal design; legislated requirements for game platforms; compliance evaluation; challenging game mechanics and access
3. Game-Related Health and Safety
    a. Injuries in Play: ways of mitigating common upper body injuries, such as repetitive strain injury; exercise psychology and physiotherapy in eSports
    b. Risk Assessment for Events and Manufacturing: control of substances hazardous to health (COSHH); fire safety; electrical and electronics safety; risk assessment for games and game events; risk assessment for manufacturing
    c. Mental Health: motivation to play; gamification and gameful design; game psychology – internet gaming disorder

4. Platform Hardware Supply Chain and Sustainability
    a. Platform Lifecycle: platform composition – materials, assembly; mineral excavation and processing; power usage; recycling; planned obsolescence.
    b. Modern Slavery: supply chains; forced labor and civil rights; working conditions; detection and remission; certification bodies and charitable endeavors.
5. Representation in the Media and Industry
    a. Inclusion: identity and identification; inclusion of a broad range of characters for diverse audiences; media representation and its effects; media literacy; content analysis; stereotyping; sexualization
    b. Equality: histories and controversies, such as gamergate, quality of life in the industry, professional discourse and conduct in business contexts, pathways to game development careers, social mobility, the experience of developers from different backgrounds and identities, gender, and technology

Illustrative Learning Outcomes:

Non-core:

1. Discuss how creators can protect their intellectual property.
2. Identify common pitfalls in game interfaces that exclude players with impaired or non-functional vision.
3. Describe how heuristic evaluation can be used to identify usability problems in game interfaces.
4. Explain why upper body injuries are common in eSports.
5. Discuss how to reform characters and dialogues in a scene to reduce stereotype threat.
6. Illustrate how the portrayal of race in a game can influence the risk of social exclusion in the associated online community around the game.
7. Modify a policy for a LAN party event to include mitigations that lower the risk of fire.
8. Design a gamification strategy to motivate serious play for an awareness-raising game.
9. Analyze the role of company hiring policies and advocacy on social mobility.
10. Assess the appropriateness of two manufacturers for producing a new game console.
11. Compare options for open-source licensing of a game development tool.
12. Recommend changes to a specific game interface to improve access to players who are deaf or whose hearing is otherwise impaired.
13. Discuss whether games are addictive.
14. Suggest how the portrayal of women in video games influences how players perceive members of those groups.
15. Create a video game that successfully advocates for climate science.

SPD-SEP/Robotics

Non-core:

1. Fairness, transparency, and accountability in robotic algorithms
2. Mitigating biases in robot decision-making
3. Public safety in shared spaces with robots
4. Compliance with data protection laws
5. Patient consent and trust in medical robots

Illustrative Learning Outcomes:
Non-core:
1. Identify instances of bias in robotic algorithms and propose strategies to mitigate them.
2. Evaluate and critique robotic systems for ethical and fairness considerations, suggesting improvements where necessary.
3. Analyze real-world examples of biases in robot decision-making and develop strategies to reduce bias in robotic systems.
4. Assess the potential risks associated with robots in public spaces and propose safety measures to mitigate those risks.
5. Evaluate the impact of patient consent and trust on the effectiveness of medical robot deployments in healthcare contexts.

SPD-SEP/Interactive

Non-core:
1. Ethical guidelines when using AI models to assist in journalism and content creation
2. Accountability for AI-generated outputs
3. Behavior among prompt programmers and AI developers
4. Trust with the public when using AI models

Illustrative Learning Outcomes:
Non-core:
1. Indicate a framework for accountability in AI model deployment, including clear documentation and attribution.
2. Discuss ethical codes of conduct and professional standards relevant to prompt programming and AI development.
3. Create communication plans and materials to educate the public about AI capabilities, limitations, and ethical safeguards.

Professional Dispositions

- **Self-Directed**: Students should be able to learn new platforms and languages with a growth-oriented mindset and thrive in dynamic environments, while continually enhancing skills.
- **Inventive:** Students should demonstrate excellence in designing software architecture within unconventional constraints, emphasizing adaptability and creative problem-solving for innovative solutions.
- **Adaptable**: Students should adapt to diverse challenges, showing resilience, open-mindedness, and a proactive approach to changing requirements and constraints.

Mathematics Requirements

Required:
- MSF-Discrete

Desired: